

TP Kubernetes avec Minikube

Déploiement et gestion d'une application containerisée

Date : 06/01/2026

Nom et Prénom : Achouchi Rayane

Projet GitHub:

<https://github.com/charroux/kubernetes-minikube>

Image Docker utilisée:

saphir10036/name-service (DockerHub)

1. Introduction

Ce TP a pour objectif de démontrer les capacités de Kubernetes en utilisant Minikube. Nous allons :

- Créer un déploiement Kubernetes à partir d'une image Docker
- Exposer l'application via un service
- Mettre en place du load balancing avec plusieurs réplicas
- Effectuer des mises à jour progressives (rolling updates)

2. Prérequis

- Minikube installé et démarré
- kubectl installé
- Docker installé (pour tester l'image en local)
- Accès à l'image Docker : saphir10036/name-service sur DockerHub

3. Étape 1 : Vérifier que Minikube est démarré

Instructions

1. Ouvrez un terminal
2. Tapez la commande :
`minikube status`
3. Vérifiez que le statut affiche "Running"

Capture d'écran attendue

Screenshot 1: Résultat de 'minikube status'

```
→ ~ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

4. Étape 2 : Créer un déploiement Kubernetes

Instructions

4. Dans le même terminal, tapez :
`kubectl create deployment name-service --image=saphir10036/name-service:latest`
5. Attendez quelques secondes que le pod se lance
6. Vérifiez avec :
`kubectl get pods`

Capture d'écran attendue

Screenshot 2: Résultat de 'kubectl get pods'

```
→ ~ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
name-service-6484bfc99b-nklx4	1/1	Running	0	19s

7. Obtenez plus de détails avec:
`kubectl describe pods`

Capture d'écran attendue

Screenshot 3: Résultat de 'kubectl describe pods'

```

➔ ~ kubectl describe pods
Name:          name-service-6484bfc99b-nklx4
Namespace:     default
Priority:       0
Service Account: default
Node:          minikube/192.168.49.2
Start Time:    Tue, 06 Jan 2026 17:29:03 +0100
Labels:        app=name-service
               pod-template-hash=6484bfc99b
Annotations:   <none>
Status:        Running
IP:            10.244.0.3
IPs:           IP: 10.244.0.3
Controlled By: ReplicaSet/name-service-6484bfc99b
Containers:
  name-service:
    Container ID:  docker://9e51cfb4b586a3f3c4c33dcc5955ad31761c3785439ff88b8772484728478ca5
    Image:         saphir10036/name-service:latest
    Image ID:      docker-pullable://saphir10036/name-service@sha256:a3d935335b36faff375ecbc546b28dae9f909f38b8ea1711bfcac89029d07a5
    Port:          <none>
    Host Port:     <none>
    State:         Running
      Started:     Tue, 06 Jan 2026 17:29:14 +0100
    Ready:         True
    Restart Count: 0
    Environment:   <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-pf9zk (ro)
Conditions:
  Type                               Status
  PodReadyToStartContainers         True
  Initialized                       True
  Ready                             True
  ContainersReady                   True

```

5. Étape 3 : Exposer le déploiement via un service (NodePort)

Instructions

8. Tapez la commande pour exposer le service en NodePort:
kubectl expose deployment name-service --type=NodePort --port=8080
9. Récupérez l'URL pour accéder au service :
minikube service name-service --url
10. Notez l'URL affichée (ex : http://192.168.99.100:32000)
11. Vérifiez le service créé :
kubectl get services

Capture d'écran attendue

Screenshot 4: Résultat de 'kubectl get services'

```

➔ ~ kubectl get services

```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	13m
name-service	NodePort	10.106.110.157	<none>	8080:30501/TCP	2m39s

```

➔ ~ minikube service name-service --url
http://127.0.0.1:39457

```

12. Ouvrez un navigateur et accédez à l'URL obtenue (ex: http://192.168.99.100:32000)

Capture d'écran attendue

Screenshot 5: Page de l'application dans le navigateur

6. Étape 4 : Mise en place du load balancing (scaling)

Instructions

13. Vérifiez le nombre de réplicas actuels :

```
kubectl get deployments
```

14. Mettez à l'échelle le déploiement pour avoir 2 instances :

```
kubectl scale --replicas=2 deployment/name-service
```

15. Vérifiez que 2 pods sont maintenant créés :

```
kubectl get pods
```

Capture d'écran attendue

Screenshot 6: Résultat de 'kubectl get pods' avec 2 replicas

```
→ ~ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
name-service-6484bfc99b-4bgnm      1/1     Running   0           7s
name-service-6484bfc99b-nklx4      1/1     Running   0          14m
```

16. Vérifiez le déploiement avec :

```
kubectl get deployments
```

Capture d'écran attendue

Screenshot 7: Résultat de 'kubectl get deployments'

```
→ ~ kubectl get deployments
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
name-service  2/2     2             2           15m
→ ~
```

7. Étape 5 : Service de type LoadBalancer

Instructions

17. Supprimez le service NodePort actuel :

```
kubectl delete service name-service
```

18. Créez un nouveau service de type LoadBalancer:

```
kubectl expose deployment name-service --type=LoadBalancer --port=8080
```

19. Obtenez l'URL du service LoadBalancer:

```
minikube service name-service --url
```

20. Vérifiez le service :

```
kubectl get services
```

Capture d'écran attendue

Screenshot 8 : Service LoadBalancer

```
→ ~ kubectl get services
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	23m
name-service	LoadBalancer	10.107.45.221	<pending>	8080:32425/TCP	9s

21. Testez l'accès dans le navigateur avec la nouvelle URL

Capture d'écran attendue

Screenshot 9 : Application accessible via LoadBalancer

Montrant l'application fonctionnant via le LoadBalancer

8. Étape 6 : Rolling Updates

Instructions

Note : Cette étape suppose que vous avez une autre version de l'image Docker disponible (ex: :latest où : v2). Si ce n'est pas le cas, vous pouvez démontrer le concept en changeant l'image vers une autre version ou ignorer cette étape.

22. Pour faire un rolling update, tapez :

```
kubectl set image deployments/name-service name-service=saphir10036/name-service:latest
```

23. Regardez l'état du déploiement pendant la mise à jour :

```
kubectl rollout status deployments/name-service
```

24. Attendez jusqu'à voir "successfully rolled out"

Capture d'écran attendue

Screenshot 10 : Rolling update en cours ou terminé

```
→ ~ kubectl set image deployments/name-service name-service=saphir10036/name-service:latest
→ ~ kubectl rollout status deployments/name-service
deployment "name-service" successfully rolled out
```

25. Pour revenir à la version précédente (rollback), tapez :

```
kubectl rollout undo deployments/name-service
```

26. Vérifiez que le rollback est effectué

9. Conclusion

Vous venez de démontrer les fonctionnalités principales de Kubernetes:

- Déploiement d'applications containerisées
- Exposition via des services (NodePort et LoadBalancer)
- Scaling automatique des pods
- Rolling updates et rollbacks

Pour nettoyer les ressources Kubernetes, vous pouvez utiliser :

```
kubectl delete services name-service  
kubectl delete deployment name-service
```

10. Récapitulatif des captures d'écran

N°	Description
1	Vérification du statut Minikube
2	Pods créés après le déploiement
3	Détails des pods
4	Service NodePort créé
5	Application accessible via NodePort
6	Pods après scaling (2 replicas)
7	Déploiement avec 2/2 replicas ready
8	Service LoadBalancer
9	Application accessible via LoadBalancer
10	Rolling update en cours ou terminé